

Competitive Programming

Section 36: Fenwick Trees

Competitive Programming Course

2026-07-10

Table of contents

1	Fenwick trees without black boxes	3
1.1	Building intuition with one update	4
2	Coordinate compression with Fenwick	5
3	Difference arrays and Fenwick	6
4	Checklist	7
5	Minimal templates	8
6	Local exercises	9
7	How to use the checker	10
8	Contest decisions, invariants, and traces	11
8.1	Trace: point add and range sum	11
8.2	Trace: inversions with duplicates	11
8.3	Difference trace	11
9	Language notes and proof habits	12
9.1	Formative checks	12
10	Submission workflow	13

1 Fenwick trees without black boxes

A Fenwick tree, also called a binary indexed tree, stores prefix information in an array `bit[1..n]`. The key idea is that index `i` owns a block whose length is the value of its lowest set bit:

```
lowbit(i) = i & -i

bit[i] stores the sum of:
i-lowbit(i)+1, ..., i
```

Example for `n = 8`:

i	lowbit(i)	covered segment
1	1	[1,1]
2	2	[1,2]
3	1	[3,3]
4	4	[1,4]
5	1	[5,5]
6	2	[5,6]
7	1	[7,7]
8	8	[1,8]

To compute a prefix sum `sum(i)`, repeatedly take the block ending at `i`, then jump to the index just before that block:

```
answer += bit[i]
i -= i & -i
```

This decomposes `[1,i]` into disjoint power-of-two blocks. For `i = 13`, the visited indices are:

```
13 -> 12 -> 8 -> 0
[13,13] + [9,12] + [1,8]
```

To add `delta` to position `i`, every Fenwick block containing `i` must increase by `delta`. Those block owners are found by moving upward:

```
bit[i] += delta
i += i & -i
```

Both loops touch at most one index per binary digit, so each operation costs $O(\log n)$.

1.1 Building intuition with one update

Suppose $n = 8$ and we add $+5$ at position 3.

The update visits:

```
3 -> 4 -> 8
```

Those are exactly the Fenwick blocks that contain position 3:

```
bit[3] covers [3,3]
bit[4] covers [1,4]
bit[8] covers [1,8]
```

No other block ending at another index includes position 3, so no other Fenwick entry changes.

For $\text{sum}(6)$, the query visits:

```
6 -> 4 -> 0
```

and reads:

```
bit[6] covers [5,6]
bit[4] covers [1,4]
```

Together these blocks form $[1,6]$ without overlap.

2 Coordinate compression with Fenwick

Fenwick indices must be dense integers. If values are as large as 10^9 , sort the distinct values and replace each value by its 1-based rank. Equal values receive the same rank. This keeps order:

```
x < y  iff  rank(x) < rank(y)
```

In inversion counting, when reading $a[i]$, the number of previous elements greater than it is:

```
previous_count - count(previous values <= a[i])
```

A Fenwick tree over ranks gives that count.

3 Difference arrays and Fenwick

A normal difference array has:

```
diff[1] = a[1]
diff[i] = a[i] - a[i-1]
a[i] = diff[1] + ... + diff[i]
```

Adding x to a range $[l, r]$ changes only two difference entries:

```
diff[l] += x
diff[r+1] -= x
```

If a Fenwick tree stores `diff`, range-add and point-query both become logarithmic.

4 Checklist

```
Use 1-based indices internally.  
Decide whether the tree stores values or differences.  
For range sums, answer  $\text{sum}(r) - \text{sum}(l-1)$ .  
For inversions, count strictly greater values; duplicates are not inversions.  
Use 64-bit integers for counts and sums.
```

5 Minimal templates

C++:

```
struct Fenwick {
    int n;
    vector<long long> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
    void add(int i, long long delta) {
        for (; i <= n; i += i & -i) bit[i] += delta;
    }
    long long sum(int i) const {
        long long ans = 0;
        for (; i > 0; i -= i & -i) ans += bit[i];
        return ans;
    }
    long long range_sum(int l, int r) const {
        return sum(r) - sum(l - 1);
    }
};
```

Python:

```
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)

    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i

    def sum(self, i):
        ans = 0
        while i > 0:
            ans += self.bit[i]
            i -= i & -i
        return ans
```

6 Local exercises

- A. Point Add Range Sum: Maintain an array. Operation 1 i x adds x to $a[i]$. Operation 2 l r asks for the sum of $a[l..r]$.
- B. Inversion Count: Given an array, count pairs $i < j$ with $a[i] > a[j]$. Values may repeat and may be large.
- C. Range Add Point Query: Maintain an array. Operation 1 l r x adds x to every element in $[l,r]$. Operation 2 i asks for the current value of $a[i]$.

7 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/36_fenwick_trees/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.
Exercises

Use A for point-add/range-sum indexing, B to count inversions after coordinate compression, and C to turn range additions into a difference Fenwick tree. Test index 1, the final index, and duplicate values.

8 Contest decisions, invariants, and traces

Do not choose a Fenwick tree because a problem says “array.” Choose it when the operations are additions and prefix aggregates, and both updates and queries must be fast. A plain prefix-sum array makes queries $O(1)$ but a point update $O(n)$; direct summation makes an update $O(1)$ but each range query $O(n)$. A Fenwick tree gives both in $O(\log n)$.

The invariant is precise: after any sequence of `add` calls, `bit[i]` equals the sum of the original logical values in $[i - \text{lowbit}(i) + 1, i]$. Prefix query visits disjoint blocks that partition $[1, i]$; update visits exactly the block owners whose block contains the changed position. These are the two facts to prove before trusting the loops.

8.1 Trace: point add and range sum

Start with `a=[2,1,4,3,5]`. Build by calling `add(i,a[i])`. For `add(3,+6)`, the logical array becomes `[2,1,10,3,5]`; the visited owners are `3,4,8,...`. For query `[2,4]`, compute `prefix(4)-prefix(1)=(2+1+10+3)-2=14`. Never try to make `bit[3]` mean “the value at index 3”; it is a block sum.

8.2 Trace: inversions with duplicates

For `[3,1,2,2]`, compress to ranks `[3,1,2,2]`. Read left to right. At the third item, `seen=2` and `prefix(rank 2)=1`, so exactly one prior value is greater. At the final 2, the prefix count is two, so again one value is greater. Equal values are included in the prefix and are therefore not counted as inversions. The final answer is three.

8.3 Difference trace

For `a=[10,20,30,40]`, store differences `[10,10,10,10]`. Adding `+5` on `[2,3]` changes only `diff[2] += 5` and `diff[4] -= 5`, giving `[10,15,10,5]`. Its prefix sums reconstruct `[10,25,35,40]`. This is why a single Fenwick over differences solves range-add/point-query.

9 Language notes and proof habits

Use 1-based internal indices: `lowbit(0)=0`, so an update starting at zero loops forever. In C++, use `long long` for sums and inversion counts. In Python, the integer size is safe, but buffered input and collecting answers before one final print matter for large operation counts. Coordinate compression must use `sorted(set(a))`, not `sorted(a)`, so equal values share a rank.

Before coding, say aloud which quantity `prefix(i)` represents. For normal storage it is `a[1]+...+a[i]`; for difference storage it is the *current value* at position `i`. This one sentence prevents most range/update mismatches.

Wrong approach	Why it fails	Test
use index 0	<code>lowbit</code> never advances	update/query at first position
use <code>seen-prefix(rank-1)</code> for inversions	counts equal values as greater	<code>[2,2]</code>
omit <code>r+1</code> subtraction	update leaks past the range	update <code>[2,3]</code> , query 4
store sums in <code>int</code>	totals can exceed 32 bits	many large additions

9.1 Formative checks

1. Which blocks does `prefix(13)` read? Hint: repeatedly subtract `lowbit`. Answer: owners 13,12,8, covering `[13,13]`, `[9,12]`, `[1,8]`.
2. Why is `previous - prefix(rank)` the inversion contribution? Answer: it removes every previous value $\leq$ the current value, leaving exactly $>$.
3. What two difference entries change for add -7 on `[1,r]`? Answer: `diff[1]-=7`, `diff[r+1]+=7`.

10 Submission workflow

Use a slow array model for stress tests: perform each update literally and compute each query by a loop. Compare it with the Fenwick result on small random arrays, especially negative additions, index one, index n , repeated values, and a full-range update. Then run:

```
CP_TARGET=solution python3 sections/36_fenwick_trees/check.py
```